swarm-compute: A Distributed Computing Framework

swarm-compute: A Distributed Computing Framework

Author: Yao Li **Date:** March 2026 **Repository:** https://github.com/BH3GEI/swarm-compute

---

1. Introduction

swarm-compute is a distributed computing framework that demonstrates core concepts of parallel task execution using a Map-Reduce inspired architecture. The system splits computation jobs into sub-tasks, dispatches them to a pool of stateless worker nodes, and aggregates the partial results into a final output.

The framework is designed to be extensible — new computation types can be added by implementing three functions (split, execute, aggregate) without modifying the core scheduling logic.

2. System Architecture

```
              ┌──────────┐
              │  Client  │ (Web UI / curl)
              └──────────┘
                   │ HTTP API
                   ▼
        ┌──────────────────┐
        │     Center       │
        │  (Scheduler)     │
        │                  │
        │  Submit ──► RabbitMQ (durable queue)
        │                  │
        │  JobRunner ◄─── consume
        │    │             │
        │    ├─ Split      │
        │    ├─ Health     │
        │    ├─ Dispatch   │
        │    └─ Aggregate  │
```

```
                    └──────────┬──────────┘
                               │
              ┌────┬────┬────┬────┬────┐
              ▼    ▼    ▼    ▼    ▼
            [W-1] [W-2] [W-3] [W-4] [W-5]
             POST /execute (parallel)
```

Components

Component	Technology	Role
Center	Go 1.22	Job scheduling, task splitting, result aggregation
Workers (×5)	Go 1.22	Stateless task execution
RabbitMQ	3.13	Durable message queue for job dispatching
Client	HTML/JS + nginx	Web interface for job submission and monitoring
Platform	HTML/JS + nginx	Admin interface for task type registry

3. Message Queue Design

Jobs flow through RabbitMQ as follows:

1. **Publish**: When a user submits a job via `POST /api/jobs/submit`, the Center creates a job record in memory and publishes the job ID to a durable RabbitMQ queue (`swarm_jobs`).
2. **Consume**: A pool of 8 goroutines (JobRunner) consumes job IDs from the queue. Each consumer picks a job, executes the full split → dispatch → aggregate pipeline, then sends a manual ACK to RabbitMQ.
3. **Durability**: The queue is declared as durable and messages use `DeliveryMode: Persistent`. This ensures jobs survive broker restarts.
4. **Fair dispatch**: `Qos(prefetch=1)` ensures each consumer processes one job at a time, preventing a slow job from blocking all consumers.

Why RabbitMQ over Go channels?

Aspect	Go Channel	RabbitMQ
Persistence	Lost on crash	Survives broker restart
Delivery guarantee	Best-effort	At-least-once (manual ACK)
Monitoring	None	Management UI at :15672
Scaling	Single process	Multiple Center instances

4. Task Type System

Each computation type implements three functions:

```
 SplitFunc:     input → [chunk₁, chunk₂, ..., chunkₙ]   // Center
ExecuteFunc:   chunkᵢ → partial_resultᵢ                 // Worker
AggregateFunc: [partial₁, ..., partialₙ] → result       // Center
```

Built-in Task Types

Type	Description	Split Strategy	Aggregation
word-count	Word frequency counting	Text → line chunks	Merge frequency maps
sort	Distributed merge-sort	Array → equal chunks	K-way merge
matrix-mul	Matrix multiplication	A → row blocks	Concatenate rows
pi-estimate	Monte Carlo Pi	Samples ÷ N	Sum inside/total
grep	Regex text search	Text → line chunks	Merge + sort by line
prime-count	Count primes in range	Range ÷ N	Sum counts
hash-crack	MD5 brute-force	Charset range ÷ N	First match

5. Fault Tolerance

- **Worker health checks**: Before dispatching, the scheduler pings all workers in parallel (2s timeout). Only responsive workers receive tasks.
- **Retry**: If a task fails, it is retried on a different worker (round-robin, up to `maxRetry` times).

- **Job cancellation**: Running jobs can be cancelled via API, which propagates context cancellation to all in-flight HTTP calls.
- **Graceful shutdown**: On SIGINT/SIGTERM, the server drains the job queue and persists state before exiting.

6. Benchmark Results

Tests conducted with 1–5 worker configurations on Docker Compose (same host):

Task	1 Worker	3 Workers	5 Workers	Speedup (5W)
Sort (100K)	66ms	42ms	42ms	1.57×
Pi (10M)	50ms	31ms	30ms	1.67×
Primes [1,1M]	19ms	13ms	15ms	1.27×

Key observations:

- Absolute execution times are very short (19–66ms for 1 worker), meaning the **fixed overhead** of the framework (health check pings, RabbitMQ publish/consume, HTTP serialization, JSON marshaling) dominates the total latency. This limits observable speedup.
- The best speedup is seen at 3 workers (~1.6×), after which adding workers provides diminishing returns due to overhead exceeding the parallelism benefit.
- **Amdahl's Law applies**: the sequential portions (split, aggregate, health check, RabbitMQ round-trip) create a floor of ~25-30ms that cannot be parallelized.
- To achieve near-linear speedup, the per-task computation must be **significantly larger** than the framework overhead. For example, Pi with 1 billion samples or Sort with 10M elements would show much better scaling.
- In a **real distributed environment** (workers on separate machines with network latency), the compute-to-overhead ratio would shift further toward computation, improving speedup.

7. Testing

Unit Tests (29 test cases)

- `center/server/splitter_test.go` — 7 tests (all splitter strategies)
- `center/server/aggregator_test.go` — 8 tests (all aggregator strategies + edge cases)
- `center/server/util_test.go` — 5 tests (ID generation, normalization)
- `site/server/executor_test.go` — 9 tests (all executors + error handling)

Integration Tests (11 test cases)

- 7 task types: correctness verification with known inputs
- Workers API: 5 workers registered

- Stats API: job count and duration tracking
- Job cancellation: cancel running job
- RabbitMQ: durable queue existence verification

All tests pass (see attached test records).

8. Production Features

- **Rate limiting**: Token bucket per client IP (50 req/s general, 10 req/s jobs)
- **Input validation**: Size limits per task type (sort: 10M elements, matrix: 1000×1000)
- **Structured logging**: JSON format with request ID propagation
- **Atomic persistence**: State saved via write-to-tmp + rename
- **RWMutex**: Read/write lock separation for concurrent access
- **Docker security**: Non-root containers, resource limits, health checks

9. How to Run

```
 git clone https://github.com/BH3GEI/swarm-compute.git
cd swarm-compute
docker compose build --no-cache
docker compose up -d
# Wait ~30s for RabbitMQ + Center to start
docker compose ps   # verify all healthy
```

- Web UI: http://localhost:8082/job-submit.html
- RabbitMQ Management: http://localhost:15672 (guest/guest)
- API: http://localhost:8080/api/task-types